

Warmth — Complete Architecture & Technical Reference

June 2026 · GTM Hackathon · Conference Intelligence Platform

One-liner: Warmth turns conference networking into a scored personal CRM — capture on mobile, score with ML, route to CRM or community, follow up from the web dashboard.

Table of contents

1. [Visual diagrams index](#)
 2. [Product overview](#)
 3. [Tech stack](#)
 4. [System architecture](#)
 5. [Warmth lifecycle](#)
 6. [iOS / watchOS architecture](#)
 7. [Python backend architecture](#)
 8. [Web dashboard architecture](#)
 9. [ML pipeline & routing](#)
 10. [External integrations](#)
 11. [API reference](#)
 12. [Data models](#)
 13. [End-to-end user journey](#)
 14. [Deployment & infra](#)
 15. [Security](#)
 16. [Repository map](#)
 17. [Implementation status](#)
 18. [Jira backlog](#)
-

Visual diagrams

Ten full-width Mermaid diagrams (sources in `docs/diagrams/*.mmd`): system architecture, lifecycle, ML routing, iOS shell & capture, backend ingest, web routes, integrations, user journey, actor sequence.

1. Product overview

Warmth is a conference intelligence platform for founders, sales, and GTM teams. At a conference you might meet 50+ people; Warmth helps you:

- **Prepare** — know who is worth meeting before you walk the floor
- **Capture** — record and extract intelligence during the conversation, hands-free
- **Route** — decide hot lead vs. community intro using *warmth uplift*
- **Follow up** — draft context-rich emails (Lightfern polishes in Gmail; human sends)

Three surfaces, one backend

Surface	Role	When
iOS + Apple Watch	Capture — phrase trigger, speech, on-device NLP	On the conference floor
Web dashboard	Review & manage — events, leads, warmth, follow-ups	Before/after the event
Python backend	Lifecycle pipelines, ML routing, integrations	Always-on server

Surface split: capture happens on **mobile**; review/management on the **web dashboard**. Both talk to the same FastAPI backend + data layer.

2. Tech stack

Mobile (iOS / watchOS)

Layer	Technology
Language	Swift 6.0
UI	SwiftUI, Liquid Glass design system
State	<code>@Observable</code> AppModel (composition root)
Speech	AVAudioEngine + SFSpeechRecognizer (on-device)
NLP	NaturalLanguage NER + regex relation extraction
Auth	Firebase Auth + Google Sign-In
Watch	WatchConnectivity + WidgetKit complications
Build	XcodeGen (<code>project.yml</code>), iOS 26+, bundle <code>com.warmth.gtmhackathon</code>

Web dashboard

Layer	Technology
Framework	React 18 + TypeScript
Build	Vite 5
Styling	Tailwind CSS 3, warm glassmorphism design
Routing	React Router 6
Auth	Firebase Auth
Data	Mock-backed API client (swappable to FastAPI)

Backend

Layer	Technology
Runtime	Python 3.10–3.13
API	FastAPI + Uvicorn
Validation	Pydantic v2
Package mgr	uv / hatchling
Database	Firebase Firestore (prod path) + in-memory demo store
Secrets	Google Secret Manager + <code>.env</code>
ML	WarmthModel, LeadScorer, LeadClusterer (stub contracts)
Workers	Passive listener (Tavily), GCP Pub/Sub (planned)

Infrastructure (target)

Service	Provider
API hosting	GCP Cloud Run
Database	Firebase Firestore
Messaging	GCP Pub/Sub
Scheduling	GCP Cloud Scheduler
Web hosting	Firebase Hosting / Vercel
iOS distribution	TestFlight → App Store

3. System architecture

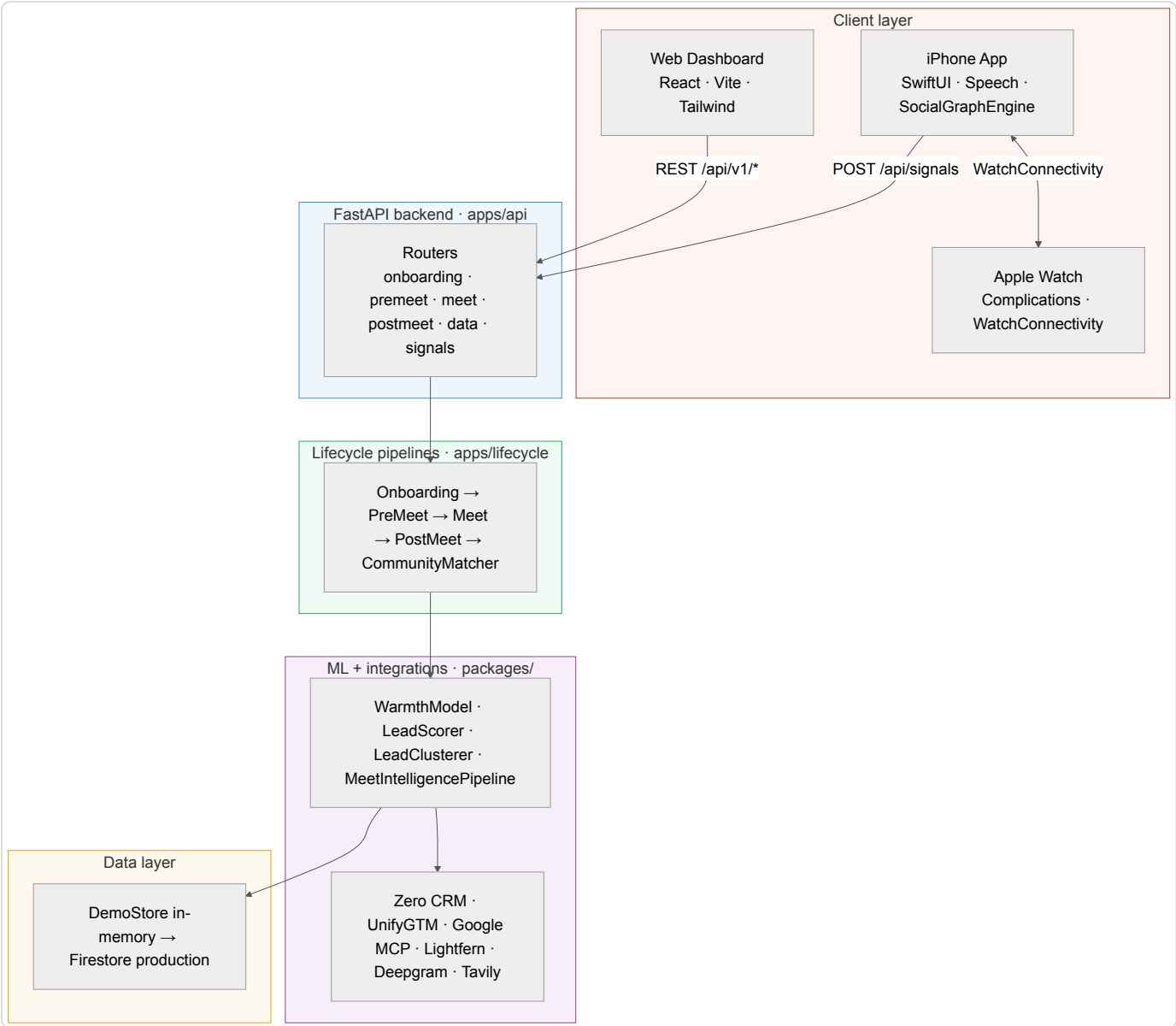
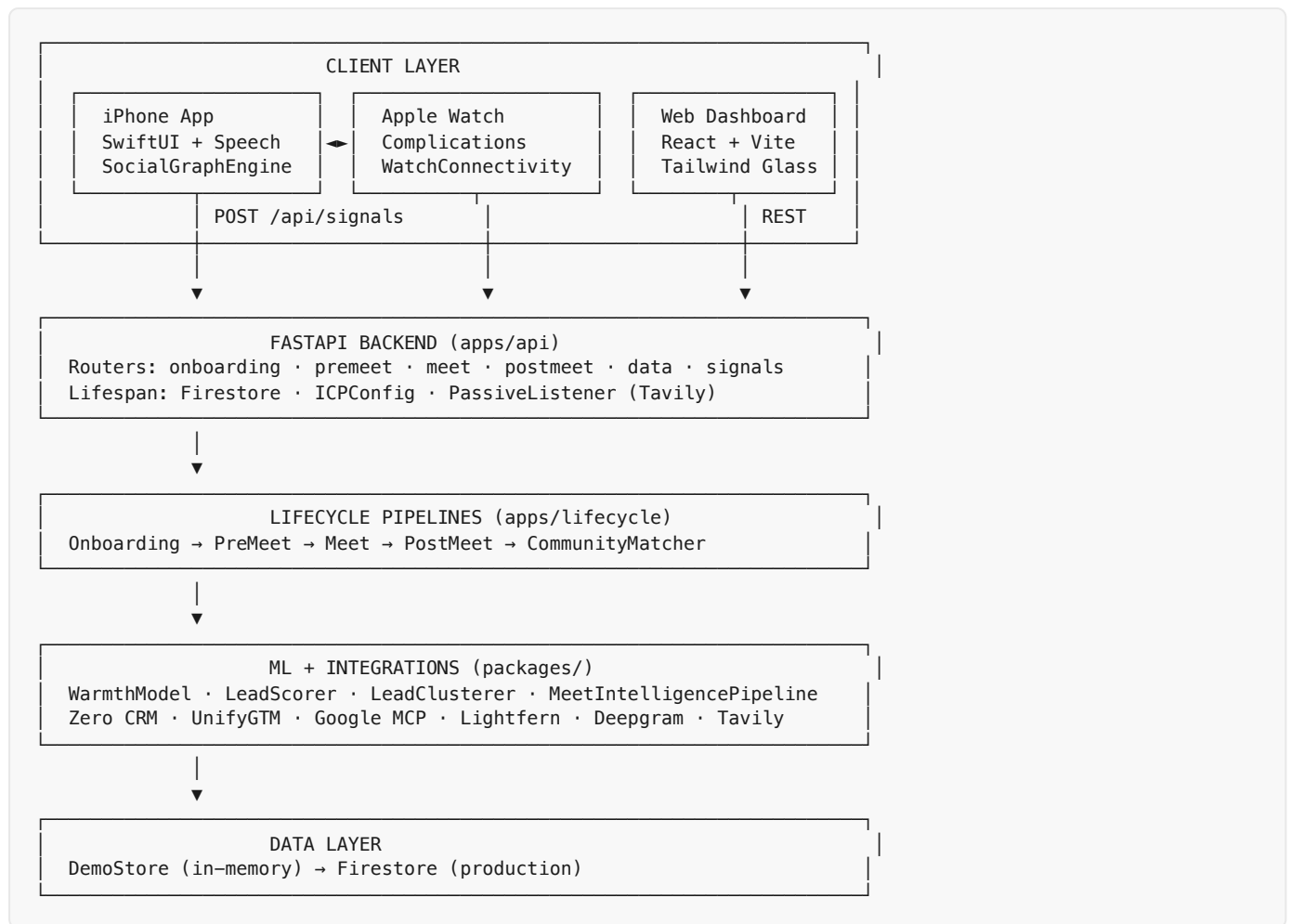


Figure 1 — System architecture — Client layer, FastAPI routers, lifecycle pipelines, ML integrations, and data store



4. Warmth lifecycle

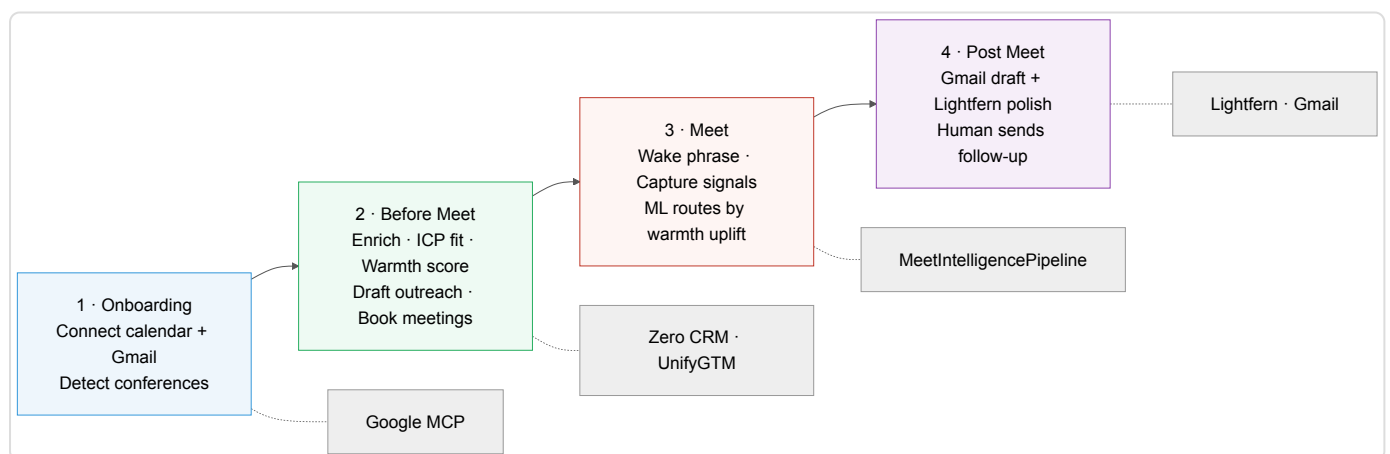
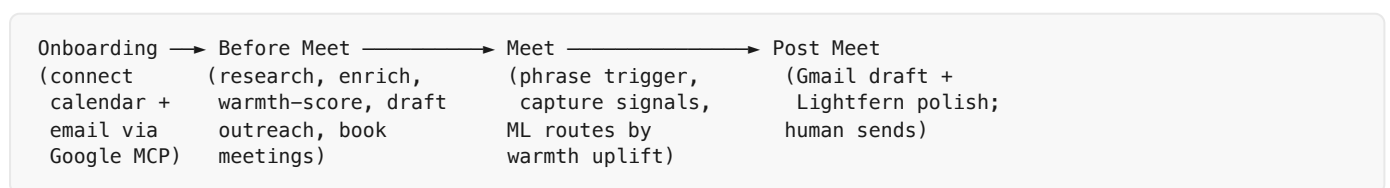


Figure 2 — Warmth lifecycle — Four lifecycle stages and their primary integrations



Stage details

Stage	What happens	Code
Onboarding	Connect Google Calendar + Gmail; detect conferences	apps/lifecycle/onboarding.py
Before Meet	Build attendee list, UnifyGTM enrich, Zero ICP fit, predict warmth, draft outreach	apps/lifecycle/premeet.py
Meet	Capture signals → ML pipeline → route CRM or community	apps/lifecycle/meet.py , packages/ml/pipeline.py
Post Meet	Gmail draft with full context; Lightfern polishes; user sends	apps/lifecycle/postmeet.py

Routing decision (core product logic)

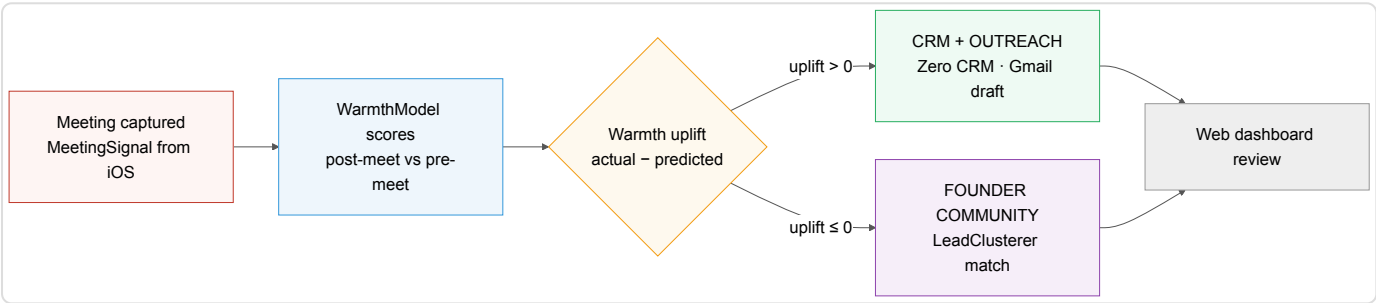
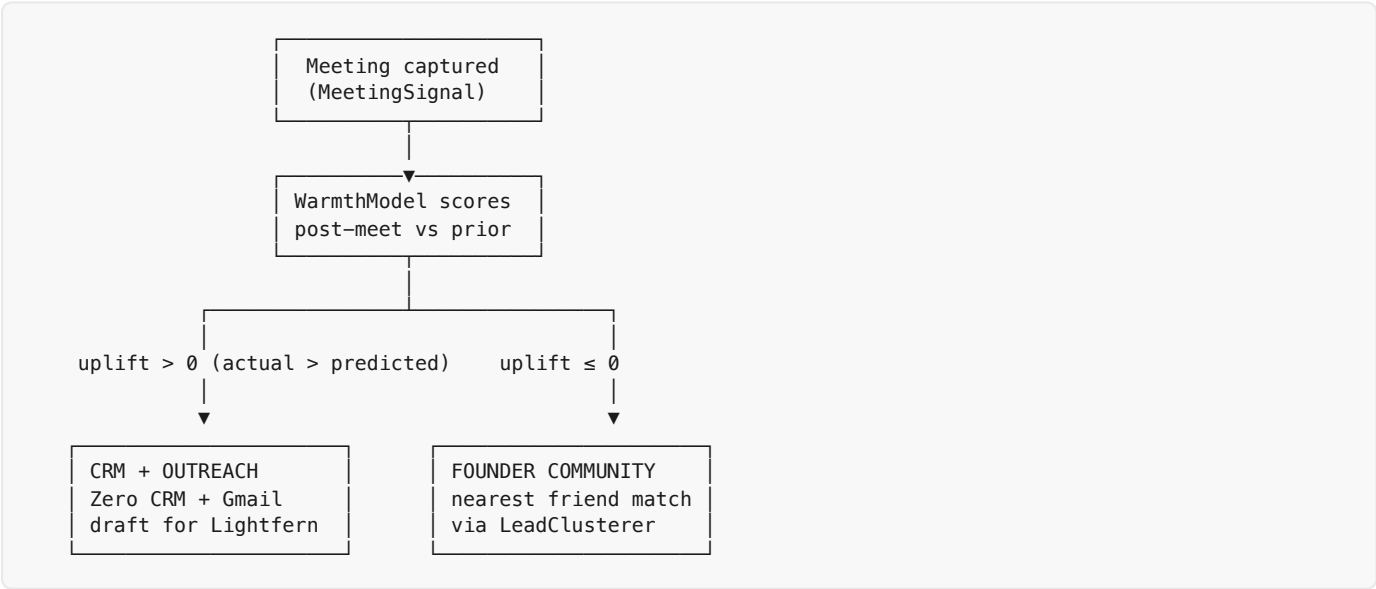


Figure 3 — ML routing decision — Warmth uplift drives CRM outreach vs founder community routing



Data ownership: ICP fit = **Zero CRM** · Enrichment = **UnifyGTM** · Warmth scoring = **Warmth ML**

5. iOS / watchOS architecture

App shell

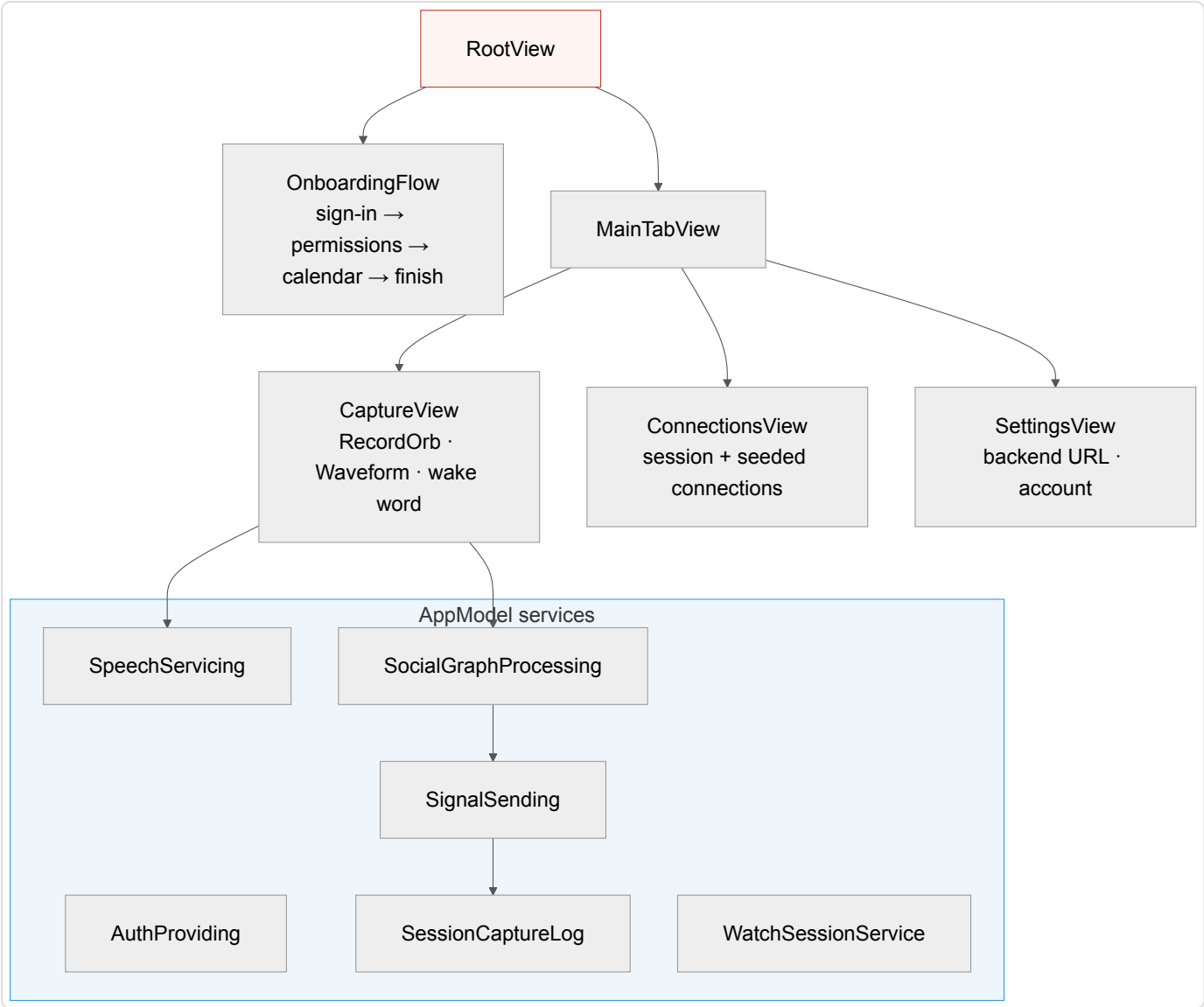


Figure 4 — iOS app shell — SwiftUI navigation shell and AppModel service wiring

```
RootView
├─ OnboardingFlow (sign-in → permissions → calendar → finish)
└─ MainTabView
    ├─ CaptureView      (RecordOrb, Waveform, wake word)
    ├─ ConnectionsView  (session + seeded connections)
    └─ SettingsView     (backend URL, account)
```

Composition root: AppModel

Service	Responsibility
AuthProviding	Firebase Auth, ID tokens for signals
SpeechServicing	AVAudioEngine → wake word → SFSpeechRecognizer
SocialGraphProcessing	On-device NLP → PersonNode
SignalSending	POST CapturedSignal to backend + retry queue
SessionCaptureLog	In-memory session people (merge by name)
WatchSessionService	iPhone ↔ Watch state sync
SettingsStore	Onboarding flag, base URL

Capture sequence

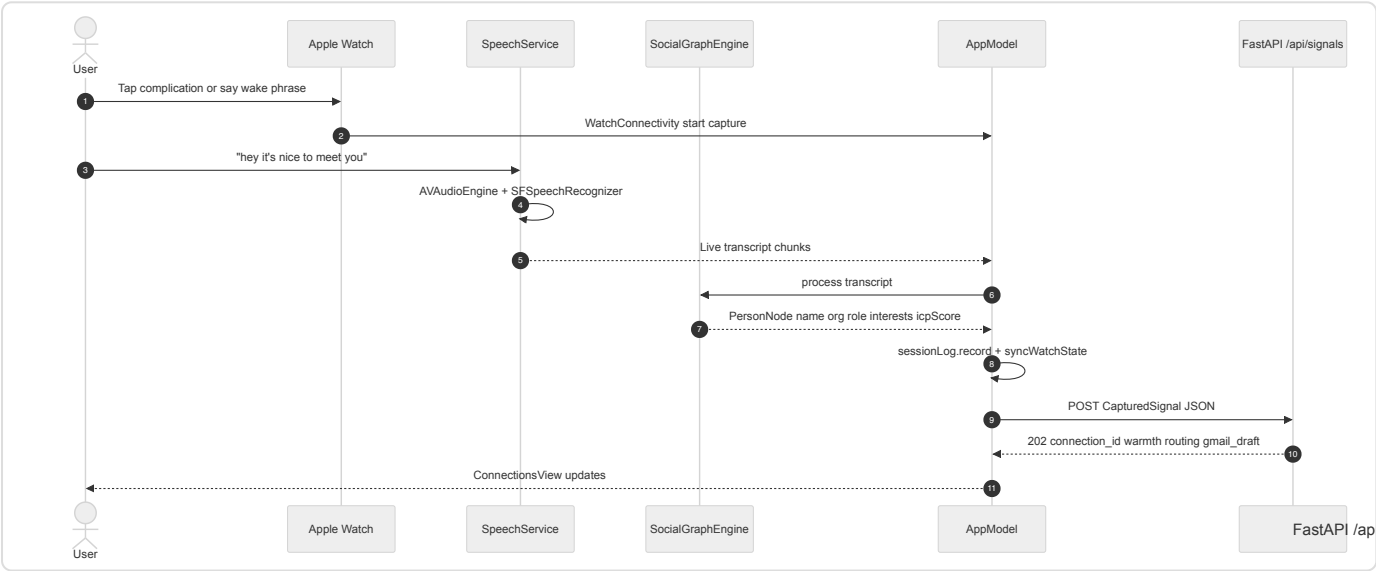


Figure 5 — iOS capture sequence — End-to-end capture flow from wake phrase to backend signal POST

```
User says "hey it's nice to meet you" (or taps Record / Watch)
↓
SpeechService.startRecording()
  | live transcript (on-device)
  ↓
SocialGraphEngine.process(transcript)
  | PersonNode: name, org, role, interests, icpScore, relations
  ↓
AppModel.capturePerson()
  ├── sessionLog.record(node)
  └── signalClient.send(CapturedSignal) → POST /api/signals
  ↓
syncWatchState() → Watch complication updates
```

SocialGraphEngine (on-device NLP)

- **NER:** NLTagger → PersonalName, OrganizationName
- **Relations:** regex SPO — works_at, founded, leads, interested_in

- **ICP score:** keyword proximity 0–100 (RevOps, GTM, SaaS, ML, etc.)
- **Output:** `PersonNode` → encoded as `CapturedSignal`

Watch targets

- `WarmthWatch` — remote start/stop capture
- `WarmthWatchWidget` — complications showing recording state + last person
- `WatchConnectivity` — bidirectional messages via `WatchSessionService`

6. Python backend architecture

FastAPI entry (`apps/api/main.py`)

- Lifespan: `FirestoreClient`, `ICPConfig`, `PassiveListener` (Tavily)
- CORS for web dashboard origins
- Health: `GET /health`

Routers

Router	Prefix / path	Purpose
onboarding	<code>/api/v1</code>	Connect calendar, list events
premeet	<code>/api/v1</code>	Run pre-meet pipeline, list leads
meet	<code>/api/v1/meet/signals</code>	Process meeting signals, return routing
postmeet	<code>/api/v1</code>	Follow-up draft generation
data	<code>/api/v1/dashboard</code> , connections, leads	Web dashboard reads
signals	<code>POST /api/signals</code>	iOS ingress (<code>CapturedSignal</code>)
conferences	conference-specific	Conference metadata

Signal ingest flow (`apps/lifecycle/signal_ingest.py`)

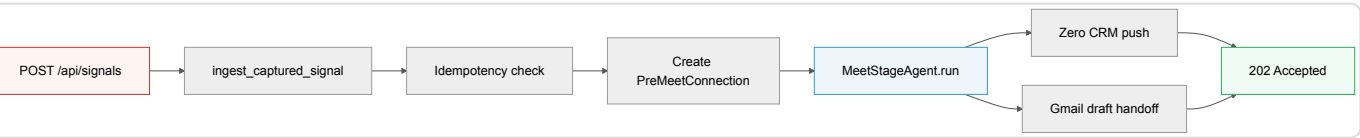
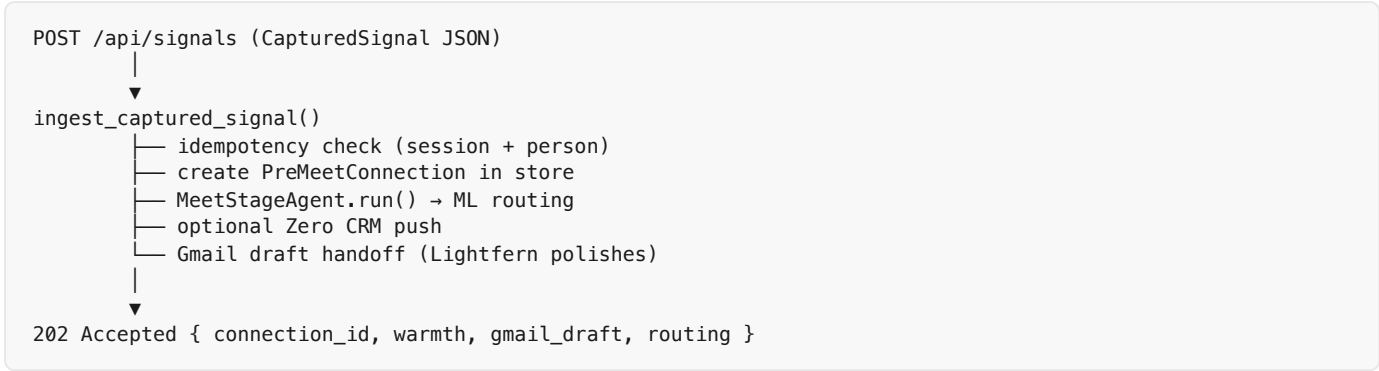


Figure 6 — Backend signal ingest — iOS signal ingress through idempotency, ML routing, and Gmail draft handoff



Demo store (apps/api/store.py)

Seeded with **SaaStr Annual 2026**, 3 attendees (Maya Chen, Diego Alvarez, Priya Nair), warmth scores, community members. Powers API + web without external keys.

7. Web dashboard architecture

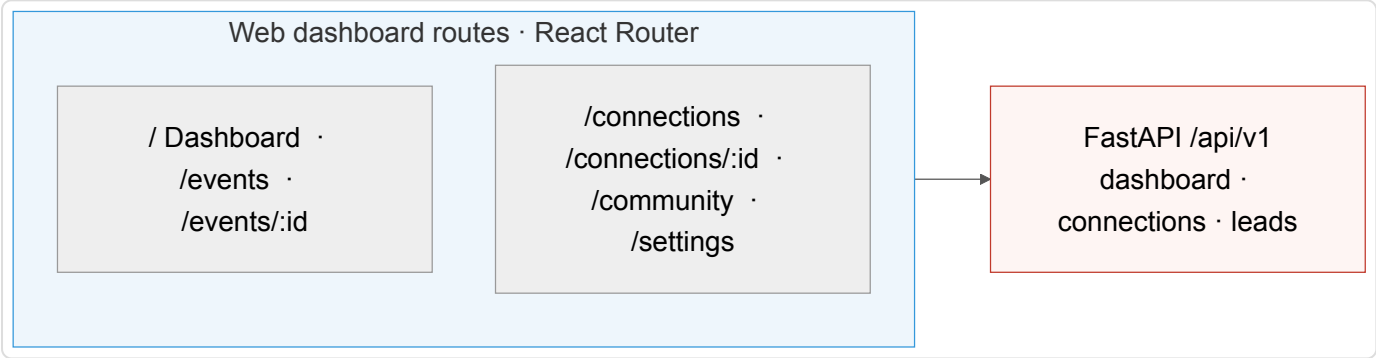


Figure 7 — Web dashboard routes — React Router pages and their FastAPI data dependencies

Routes (web/src/App.tsx)

Route	Page	Purpose
<code>/</code>	Dashboard	Stats, upcoming events, top leads
<code>/events</code>	Events	Detected conferences
<code>/events/:id</code>	PreMeet	Ranked leads, outreach actions
<code>/connections</code>	Connections	All connections by warmth
<code>/connections/:id</code>	ConnectionDetail	Scores, routing, follow-up
<code>/community</code>	Community	Founder network matches
<code>/settings</code>	Settings	Account preferences

Design system

- Warm glassmorphism (`Glass.tsx` , `WarmthBadge` , `ConnectionWeb` graph)
- Space Grotesk typography

- Mock data layer (`mockData.ts`) — swap to `VITE_API_BASE_URL` for live backend

8. ML pipeline & routing

Models (`packages/ml/`)

Model	Input	Output
<code>WarmthModel</code>	ICP fit (Zero) + enrichment (Unify) + engagement signals	predicted_score, actual_score, uplift, band
<code>LeadScorer</code>	ICP config + connection attrs	Intent score; fallback ICP fit
<code>LeadClusterer</code>	MeetingSignal + community list	cluster_id, nearest neighbors
<code>MeetIntelligencePipeline</code>	MeetingSignal + prior WarmthScore	RoutingDecision (CRM vs community)

WarmthScore fields

- `icp_score` — 0–100 from Zero CRM
- `warmth_score` — 0–100 relational/intent warmth
- `predicted_score` — pre-meet combined
- `actual_score` — post-meet combined
- `uplift` = actual – predicted → drives routing
- `band` — cold / warm / hot

9. External integrations

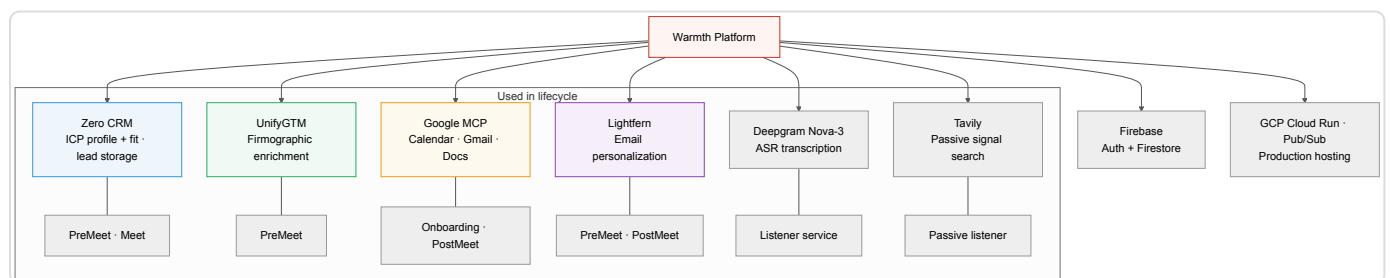


Figure 8 — Integration map — Third-party services and which lifecycle stages consume them

Integration	Owns	Used in	Status
Zero CRM	ICP profile + fit, lead storage	PreMeet, Meet	Stub + optional client
UnifyGTM	Firmographic enrichment	PreMeet	Stub
Google MCP	Calendar, Gmail, Docs	Onboarding, PostMeet	OAuth path documented
Lightfern	Email personalization	PreMeet, PostMeet	Webhook stub
Deepgram	ASR transcription	Listener service	Optional dep
Tavily	Signal search	Passive listener	Wired in lifespan
Firebase	Auth + Firestore	iOS, Web, Backend	Partial
Cursor SDK	Agent extraction	MeetStageAgent	Optional

Email policy: Warmth creates Gmail *drafts*. Lightfern polishes inside Gmail. The human always sends — no auto-send.

10. API reference

Lifecycle (v1)

Method	Path	Stage
POST	/api/v1/connect	Onboarding
GET	/api/v1/events	Onboarding
POST	/api/v1/events/{id}/premeet	Before meet
GET	/api/v1/events/{id}/leads	Before meet
POST	/api/v1/meet/signals	Meet → routing decision
POST	/api/v1/connections/{id}/followup	Post meet
GET	/api/v1/dashboard	Web reads
GET	/api/v1/connections	Web reads
GET	/api/v1/leads	Web reads

iOS ingress

```
POST /api/signals
Content-Type: application/json

{
  "user": { "uid": "...", "id_token": "..." },
  "session_id": "UUID",
  "captured_at": "2026-06-20T12:00:00Z",
  "person": { "name": "Maya Chen", "org": "NorthWind Labs", "role": "VP RevOps" },
  "relations": [{ "subject": "Maya", "predicate": "works_at", "object": "NorthWind" }],
  "interests": ["RevOps", "pipeline"],
  "icp_keyword_score": 88,
  "transcript_excerpt": "...",
  "device": { "model": "iPhone", "os": "iOS 26" }
}

→ 202 { status, connection_id, warmth, gmail_draft, routing }
```

11. Data models

Core entities

Model	Description
DetectedEvent	Conference from calendar detection
PreMeetConnection	Attendee before meeting (enriched)
MeetingSignal	Live capture: name, interests, topic_time, takeaways
WarmthScore	ICP + warmth + predicted/actual/uplift
Lead	CRM-ready lead record
CapturedSignal	iOS wire format (Swift + Pydantic)
RoutingDecision	CRM_AND_OUTREACH or FOUNDER_COMMUNITY

12. End-to-end user journey

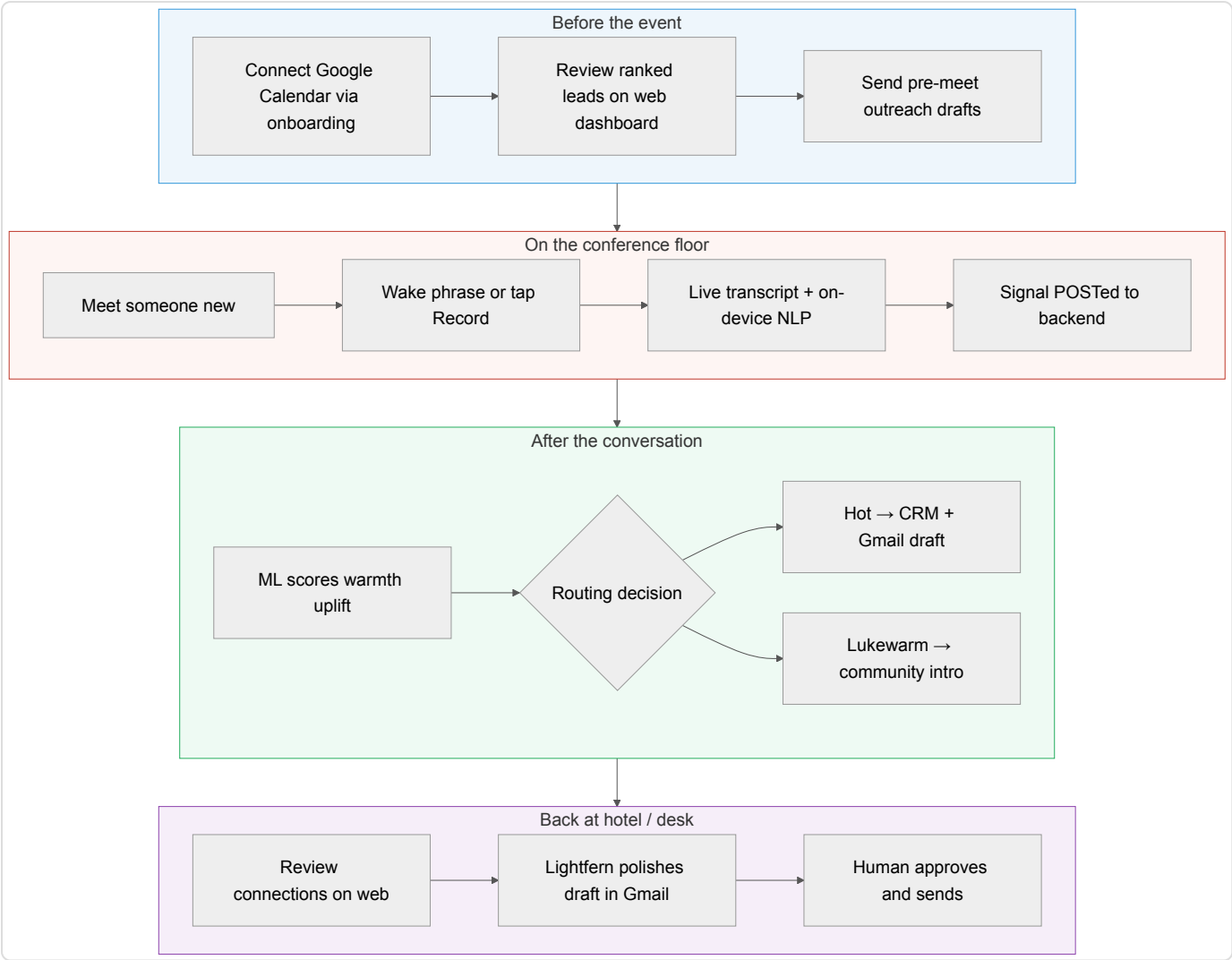


Figure 9 — User journey — User journey swimlanes

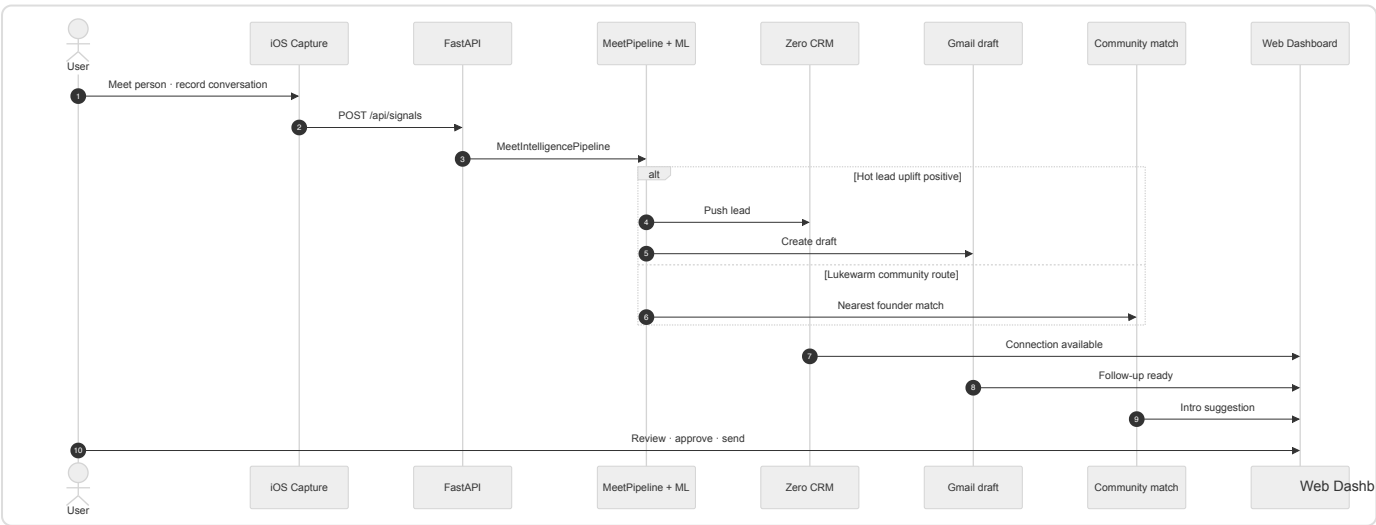
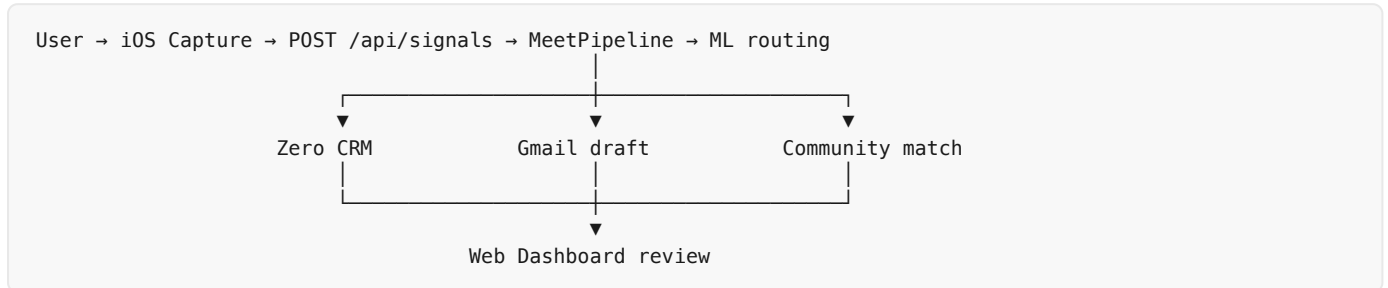


Figure 10 — Cross-system sequence — Capture → ML routing → CRM / Gmail / community → web review

BEFORE EVENT	ON THE FLOOR	AFTER
Connect calendar →	Meet someone →	Review connections
Review ranked	Say wake phrase	Approve Gmail draft
leads on web	Live transcript	Lightfern polishes
Send outreach	NLP extracts person	Human sends email
drafts	Backend routes hot/cold	

Actor sequence



13. Deployment & infrastructure

Local development

```

# Backend
uv sync && uv run uvicorn apps.api.main:app --reload --port 8000

# Web
cd web && npm install && npm run dev # http://localhost:5173

# iOS
cd iOS/Warmth-iOS && xcodegen generate
xcodebuild build -scheme Warmth -destination 'platform=iOS Simulator,name=iPhone 17 Pro'

```

Production target

- **API:** GCP Cloud Run
- **DB:** Firebase Firestore
- **Secrets:** Google Secret Manager
- **Web:** Static deploy (Vercel / Firebase Hosting)
- **iOS:** TestFlight → App Store

14. Security

- Firebase Auth + ID tokens on iOS signal payloads
- Google Secret Manager for API keys (Deepgram, Zero, Unify, Tavily)
- CORS restricted via `WEB_ALLOWED_ORIGINS`
- On-device speech + NLP (privacy on capture)
- Gmail drafts only — no auto-send without human action

15. Repository map

gtmhackathon/

apps/

api/

FastAPI + routers + demo store

lifecycle/

Onboarding, PreMeet, Meet, PostMeet, signal_ingest

listener/

Passive Tavily listener

agent/

MeetStageAgent (Cursor SDK extraction)

packages/

core/

Pydantic models + schemas

ml/

WarmthModel, pipeline, clustering

integrations/

Zero, Unify, Google MCP, Lightfern, Tavily

infra/firebase/

FirestoreClient

web/

React dashboard

iOS/Warmth-iOS/

iOS + Watch + Widget targets

tests/

Python unit tests

docs/confluence/

This documentation

16. Implementation status

Area	Status
iOS capture pipeline	Done
iOS UI (Capture, Connections, Onboarding)	Done
Apple Watch + complications	Done
Web dashboard (mock data)	Done
Backend lifecycle + demo store	Done
Signal ingest + Gmail draft handoff	Done
Wake word (real provider)	In Progress
Firebase Auth E2E	In Progress
ML models (real training)	In Progress
Zero / Unify / Google MCP live	To Do
Firestore persistence	To Do
GCP production deploy	To Do

17. Jira backlog

Full sprint backlog tracked in Jira project **KAN**:

- **Epic:** KAN-1 — Warmth — Conference Intelligence Platform
- **Board:** KAN Board

Child issues KAN-2 through KAN-32 cover Done / In Progress / To Do work across iOS, backend, integrations, and deploy.

Generated from the gtmhackathon repository · branch cursor/warmth-ios-enhancements

Generated from the gtmhackathon repository · branch cursor/warmth-ios-enhancements